The Ultimate LLM Prompting Guide

Maximizing Response Quality from Claude and Any LLM

Based on Anthropic's Official Prompt Engineering Documentation • Claude 4.x Models

---

Prompt engineering is the art and science of crafting inputs that consistently produce the highest-quality, most relevant outputs from a large language model. This guide distills every technique from Anthropic's official documentation — analysed, ranked, and translated into actionable key points — so you can get maximum value from any prompt, for research, instructions, or complex tasks.

BEFORE YOU ENGINEER ANY PROMPT

These three foundations must be in place before applying any technique. Skipping them leads to optimising for the wrong outcome.

- **Step 1 —** Define what success looks like.
  - Write down what the ideal output looks like — its format, depth, tone, and completeness.
  - Without a clear target, you cannot tell whether your prompt is working.
- **Step 2 —** Create empirical tests you can run repeatedly.
  - Draft at least 5–10 representative test cases that reflect real use.
  - Use consistent evaluation criteria: accuracy, relevance, format, tone.
- **Step 3 —** Draft your first prompt, then improve it.
  - Even a rough first draft gives you a baseline to test against.
  - Use Anthropic's Prompt Generator tool if starting from a blank page.

💡 **Core principle:** Think of the LLM as a highly capable but brand-new employee with no knowledge of your specific context, norms, or preferences. The more precisely you explain what you want, the better the response will be.

AT A GLANCE: ALL 8 TECHNIQUES RANKED

The table below summarises all techniques in order of broadest applicability. Apply them in this order when troubleshooting underperforming prompts.

#	Technique	What It Does	Best Used For	Impact
1	**Be Clear & Direct**	Removes ambiguity from instructions	All prompts — start here	★★★★★
2	**Multishot Examples**	Shows desired format/style via samples	Structured output, tone matching	★★★★★
3	**Chain of Thought (CoT)**	Forces step-by-step reasoning	Complex analysis, math, research	★★★★★
4	**XML Tags**	Structures multi-part prompts	Long, complex, multi-section prompts	★★★★☆
5	**Role / System Prompts**	Gives Claude expert identity	Domain expertise, tone control	★★★★☆
6	**Prompt Chaining**	Breaks tasks into sequential steps	Long documents, multi-step workflows	★★★★☆
7	**Long Context Tips**	Manages large inputs effectively	Research docs, books, datasets	★★★☆☆
8	**Prompt Generator**	Auto-creates first-draft prompts	Starting from scratch	★★★☆☆

TECHNIQUE 1 — BE CLEAR AND DIRECT

Impact: Highest. This is the single most universally applicable technique.

Why it matters

Vague prompts produce vague answers. Every word of ambiguity in your prompt is a decision delegated to the model — and the model may not decide the way you want. Precision eliminates guesswork.

Key action points

- State your goal explicitly in the first sentence. Don't bury the task.
- Specify the output format upfront: paragraph, table, JSON, numbered list, etc.
- Include the intended audience: 'Explain this to a non-technical executive' vs 'Explain this to a senior software engineer.'
- Say what to EXCLUDE as well as what to include. Negative constraints are equally powerful.
- Specify length: 'In 200 words or fewer', 'In exactly 5 bullet points', 'A two-page summary.'
- If tone matters, name it: formal, conversational, persuasive, neutral, urgent.
- Use the Golden Rule: show your prompt to a colleague unfamiliar with the task. If they're confused about what to do, the model will be too.

💡 **Power move:** Replace generic openers like 'Tell me about X' with specific task verbs: Compare, Summarise, Analyse, Draft, Extract, Classify, Rank, Translate, Critique. Specific verbs produce specific outputs.

TECHNIQUE 2 — USE EXAMPLES (MULTISHOT PROMPTING)

Impact: Highest. Examples are the fastest way to align output to your exact requirements.

Why it matters

Describing what you want is often less powerful than showing what you want. Well-crafted examples communicate format, tone, depth, and style simultaneously — more efficiently than any written instruction.

Key action points

- Provide 3–5 examples for best results (hence 'multishot' or 'few-shot'). One example is a start; five examples produce significantly more consistent outputs.
- Wrap each example in XML tags for structure — use <example> or nest multiple within <examples>.
- Make examples diverse: cover edge cases, different tones, varying content — not just the 'easy' case.
- Ensure examples are high quality. Poor examples actively teach the wrong pattern; review them as carefully as you'd review final output.
- For structured outputs (JSON, tables, formatted reports), examples are especially critical — they define the exact schema or layout expected.
- Ask the model to evaluate your own examples for clarity, relevance, and diversity if you're unsure.
- Extend examples over time: as you encounter new edge cases, add them to your example set.

💡 **Advanced move:** Combine multishot examples with chain-of-thought: include not just input-output pairs, but examples that show the reasoning process. This teaches the model both the format AND the thinking approach you expect.

TECHNIQUE 3 — LET CLAUDE THINK (CHAIN OF THOUGHT)

Impact: Highest for complex tasks. Dramatically reduces reasoning errors.

Why it matters

When a model is forced to generate its final answer immediately, it skips intermediate reasoning steps — which leads to errors in logic, mathematics, multi-step analysis, and nuanced judgement. Chain of thought (CoT) forces the model to 'show its work' before concluding, which surfaces errors and improves accuracy.

Three levels of CoT — apply in order of complexity

- Basic CoT — Add 'Think step by step' to your prompt. Simple but effective for moderately complex tasks.
- Guided CoT — Outline specific reasoning steps for the model to follow. Useful when the task has a known logical structure.
  - Example: '1. Identify the key variables. 2. Assess each against the criteria. 3. Compare and rank. 4. Recommend.'
- Structured CoT — Use XML tags <thinking> and <answer> to separate reasoning from the final output. This makes it easy to extract just the conclusion when needed.

Key action points

- CRITICAL: Always instruct the model to output its thinking. If it doesn't write out the reasoning, no reasoning occurs.
- Use CoT for: complex analysis, mathematical problems, multi-factor decisions, research synthesis, document drafting, debugging.
- For very complex tasks, use extended thinking mode with a <thinking> or <scratchpad> tag, then a clean <answer> block.
- Combine CoT with multishot examples — include examples that show the reasoning process, not just the final answer.
- Control verbosity: if CoT outputs are too long, add 'Show key reasoning steps only, no more than 3 sentences of thinking before giving the final answer.'

💡 **Research & analysis use case:** For research prompts, structured CoT is transformative. Ask the model to: (1) identify what information is needed, (2) assess what is known vs uncertain, (3) reason through the evidence, and (4) state conclusions with confidence levels. This mirrors expert analytical thinking.

TECHNIQUE 4 — USE XML TAGS TO STRUCTURE PROMPTS

Impact: High for multi-part, complex prompts. Eliminates parsing errors and improves output organisation.

Why it matters

When a prompt contains instructions, context, examples, constraints, and formatting requirements all mixed together, the model must figure out which part is which. XML tags remove this ambiguity entirely — each component of the prompt is labelled, and the model processes each correctly.

Key action points

- Separate every major component of your prompt with descriptive tags.
  - Use: <instructions>, <context>, <examples>, <output_format>, <constraints>, <data>, <question>
- For CoT, use <thinking> for reasoning and <answer> for the final output — this makes extraction trivial in automated workflows.
- Nest tags logically: wrap multiple examples in <examples> with individual <example> tags inside.
- Keep tag names descriptive and consistent across all your prompts — treat them as a schema.
- Combine XML tags with other techniques for maximum power: XML + multishot + CoT is the highest-performance prompt structure available.
- In the output, ask the model to use specific XML tags — this makes parsing programmatic outputs far easier.

💡 **Template structure:** A high-performance prompt structure: <context>[background info]</context> → <instructions>[what to do]</instructions> → <examples>[input/output pairs]</examples> → <output_format>[exact format required]</output_format> → <question>[the actual task]</question>.

TECHNIQUE 5 — GIVE CLAUDE A ROLE (SYSTEM PROMPTS)

Impact: High. Transforms general assistant into a domain expert with consistent tone and focus.

Why it matters

Assigning a specific role transforms the model's default behaviour. A model told it is a senior financial analyst applies different heuristics, uses different vocabulary, and maintains different standards than a general assistant — even when answering the same question.

Key action points

- Assign the role via the system prompt (not in the user message) for cleanest separation.
- Be specific about the role: not 'You are an expert' but 'You are a senior data scientist with 15 years of experience in predictive modelling for financial services.'
- Include relevant behavioural constraints in the role: 'You always cite your reasoning', 'You always ask for clarification before proceeding', 'You never make recommendations without stating your confidence level.'
- Use role prompts to control tone: a legal analyst is concise and precise; a customer success manager is warm and solution-oriented; a technical writer is structured and jargon-free.
- For research tasks: assign the role of a domain expert who critically evaluates sources, identifies gaps, and synthesises findings — rather than a general assistant who summarises.
- Keep task-specific instructions in the user turn; reserve the system prompt for the identity, values, and persistent behaviours of the role.

💡 **Powerful pairing:** Combine role assignment with chain of thought: 'You are a senior strategy consultant. Think through this problem step by step before giving your recommendation.' This pairs domain expertise with rigorous reasoning — the combination produces significantly better analytical outputs.

TECHNIQUE 6 — CHAIN COMPLEX PROMPTS

Impact: High for long-form or multi-stage tasks. Prevents context overload and error propagation.

Why it matters

Asking one prompt to research, analyse, draft, edit, and format a complex document all at once overwhelms the model's ability to excel at any single step. Chaining breaks the task into sequential stages, where the output of one prompt becomes the input of the next — allowing the model to focus fully on each discrete sub-task.

Key action points

- Identify natural breakpoints in your task: research → outline → draft → review → format are classic stages.
- Use the output of each step as explicit context for the next: 'Here is the research summary from Step 1: [output]. Now, using only this information, draft an outline.'
- Validate each intermediate output before passing it forward. Errors in early steps compound in later ones.

- For very long documents, chain by section: generate each section independently, then combine and harmonise.
- Use consistent XML tags across chain steps so outputs are structured inputs for the next prompt.
- Apply this technique whenever a single prompt produces mediocre results on a complex task — the solution is almost always to decompose it.

💡 **Research workflow example:** Step 1: 'List the key themes in [topic] with one sentence each.' → Step 2: 'For each theme below, expand with evidence and counterarguments.' → Step 3: 'Synthesise these themes into a cohesive 500-word analysis with a clear conclusion.' Each step builds precisely on the last.

TECHNIQUE 7 — LONG CONTEXT TIPS

Impact: High when working with large documents, datasets, or research corpora.

Why it matters

Modern LLMs support very large context windows, but more context does not automatically mean better outputs. How information is positioned within a long context significantly affects what the model attends to most carefully.

Key action points

- Place the most important instructions and context at the top of the prompt, not buried in the middle of a long document.
- Put your task or question at both the beginning AND the end of a long context prompt — this 'bookends' the instruction and reinforces the goal.
- Use clear section labels and XML tags to divide long contexts into labelled segments that the model can reference.
- Instruct the model explicitly on which part of the context to use: 'Using only the financial data in the <data> section below, answer the following question…'
- For document review tasks, direct the model to extract and quote relevant sections before analysing — this ensures it engages with the source material rather than relying on general knowledge.
- When passing a very long document: summarise it first (via a prior chained prompt), then pass the summary plus targeted extracts rather than the entire document.

💡 **Position matters:** Research shows that LLMs attend more strongly to the beginning and end of a context window than the middle. For long prompts, put critical instructions at both ends — not just at the top.

TECHNIQUE 8 — USE THE PROMPT GENERATOR

Impact: Moderate. Best for creating a solid starting point when you don't know where to begin.

Anthropic's Prompt Generator (available in the Claude Console) automatically creates structured, high-quality prompt templates for specific tasks. It applies prompt engineering best practices automatically — then you refine the output rather than starting from scratch.

Key action points

- Use the generator to produce a first draft, then apply Techniques 1–7 to refine it.
- Treat the generated prompt as a template with variables — replace placeholders with your actual task parameters.
- Use the Prompt Improver tool to take an existing underperforming prompt and get a structured improved version.
- Even if you don't use the generator, read the output it produces for similar tasks — it teaches you the structure of well-engineered prompts.

THE MASTER FRAMEWORK: COMBINING ALL TECHNIQUES

For maximum output quality on any serious research, instructional, or analytical task, apply this framework. Each layer adds precision and reduces error.

1. Define success criteria and test cases before writing a single prompt.
2. Assign an expert role via system prompt (Technique 5).
3. Open your prompt with a clear, specific task statement (Technique 1).
4. Structure all components with XML tags (Technique 4).
5. Include 3–5 high-quality examples of the desired output (Technique 2).
6. Instruct the model to reason step by step before answering (Technique 3).
7. If the task is complex, break it into a chain of focused sub-prompts (Technique 6).
8. For large inputs, use positioning and labelling strategies (Technique 7).
9. Test, evaluate against your criteria, and iterate.

💡 **The single most powerful combination:** Role prompt + Chain of Thought + XML structure + Multishot examples. This four-layer approach consistently produces the highest-quality outputs across research, analysis, writing, and complex reasoning tasks. Start with clarity (Technique 1) and layer the others on top.

WHAT PROMPT ENGINEERING CANNOT FIX

Prompt engineering is powerful, but it has limits. Recognising these limits saves time and prevents misdiagnosis.

- Latency and cost — these are model selection problems, not prompt problems. Use a smaller, faster model if needed.
- Fundamental knowledge gaps — if the model lacks training data on a topic, prompting cannot create knowledge; use retrieval-augmented generation (RAG) or supply the information directly in context.
- Output inconsistency at scale — fine-tuning on curated examples is more robust than prompting alone for production systems requiring strict consistency.
- Factual hallucinations — prompting reduces but does not eliminate hallucinations; always verify high-stakes factual outputs against primary sources.

QUICK DIAGNOSTIC REFERENCE

If your prompts are producing poor results, use this checklist to identify the likely cause and the fix.

Problem	Likely Cause	Fix
Output is too generic	Prompt is vague	Technique 1: Be specific about format, length, audience
Wrong format or structure	No format guidance given	Technique 1 + 2: State format; show an example
Reasoning errors or missed steps	No CoT instruction	Technique 3: Add 'Think step by step' or guided reasoning
Model misreads multi-part prompt	No structure in prompt	Technique 4: Use XML tags to separate components
Wrong tone or level of expertise	No role assigned	Technique 5: Add system prompt with specific expert role
Task too complex, outputs mediocre	Trying to do too much at once	Technique 6: Chain into focused sub-prompts
Model ignores key info in long doc	Poor context positioning	Technique 7: Reposition instructions; bookend key points
No idea where to start	Blank page problem	Technique 8: Use Anthropic's Prompt Generator